

When Does a Migration Skill Help? A Retrospective Study of Version-Pinned Plugin Migration

Anonymous ACL submission

Abstract

Migration skills package version-specific facts, procedural guidance, and tools, but their practical utility is not guaranteed. We study one skill lineage in a community-informed benchmark of version-pinned plugin migration, covering static diagnosis and executable repair. Five historical with-skill versus no-skill comparisons show heterogeneous absolute gains: -3.1 points for the lowest-baseline configuration, $+8.7$ to $+10.7$ for three intermediate-baseline configurations, and $+3.0$ for the highest-baseline configuration. These observations form an exploratory inverted-U pattern when ordered by each group’s unaided score. The groups differ in task pools, graders, skill revisions, and execution protocols, so the pattern does not establish an effect of model capability. The negative estimate is inconclusive and infrastructure-confounded; only the glm-5.3-flash pairing is significant under both reported statistical procedures, and it uses a same-model judge. The high-baseline pairing has limited room to improve. Resource records additionally show that skill provision can accompany substantial overhead. We release task and evaluation artifacts with a reproducible paired analysis and explicit provenance limits. The resulting single-ecosystem study documents observed utility and motivates, rather than establishes, a capability-dependent hypothesis.

1 Introduction

Migrating a plugin across a breaking framework release requires identifying the target contract, locating affected code, and verifying the repaired behavior. Version-aware code generation and migration benchmarks document the difficulty of adapting to changing interfaces (Wu et al., 2024; Cheng et al., 2025). A migration *skill* packages reference knowledge with guidance on inspection, implementation, and verification. Such guidance is

attractive for agents deployed under limited resources: a reusable procedure could help them spend a fixed budget on the right actions. Reading and following that procedure also consumes resources that could otherwise be spent inspecting code or testing a repair.

Existing skill benchmarks report heterogeneous utility, including limited gains and regressions (Li et al., 2026; Han et al., 2026). However, a with-skill versus no-skill comparison bundles several interventions: access to previously unavailable facts, procedural guidance, auxiliary tools, and changes in resource consumption. It answers whether the package helps, but not whether organizing the same knowledge as a skill adds value over supplying its source documents. That distinction is especially relevant when the agent must finish within a fixed budget.

We ask: **What benefits, regressions, and resource costs accompany migration-skill provision in the available historical evaluations?** Our primary evidence consists of task-level paired comparisons within five model–execution configurations. We additionally examine whether their absolute gains display a common shape, while treating differences across configurations as exploratory rather than as a controlled capability experiment.

We study DSH, an open-source plugin framework with versioned contracts and a host CLI for testing installed plugins. Tasks combine static diagnosis with hands-on repair, including obsolete comments, unrelated failures, and stale artifacts. These conditions expose migration difficulties, but performance on them alone does not identify resistance to misleading context. The scope is retrospective and open-book within one ecosystem.

The paper contributes:

- A community-informed migration benchmark with version-pinned task artifacts, diag-

083
084
085

086
087
088
089

090
091
092
093

094
095
096

097

098
099
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116

117
118
119
120
121
122
123
124
125
126
127

128
129

nosis rubrics, and executable checks; historical task membership and development exposure are explicitly distinguished.

- A reproducible paired analysis of five historical configurations: four multi-run groups and one single-shot group, with a contaminated sixth pairing reported separately.
- An exploratory inverted-U observation in absolute score gains, together with its statistical, ceiling, grading, and infrastructure limits and the available resource records.

A source-matched control protocol is provided in Appendix A as unexecuted future work, not as a validated method or an empirical contribution.

2 Related Work

Agent skill evaluation. SkillsBench evaluates curated skills through paired conditions and deterministic verifiers (Li et al., 2026). SWE-SkillsBench uses execution-based acceptance tests and reports heterogeneous effects, including regressions associated with version-mismatched guidance (Han et al., 2026). SkillLens studies experience generation, skill extraction, and consumption, finding that utility varies across extractors and consumers (Huang et al., 2026). Agent benchmarks more broadly measure interactive task completion (Jimenez et al., 2024; Liu et al., 2024b); our tasks sit closer to the static-diagnosis end, with executable repair checks on the hands-on subset. Our focus is retrospective skill utility in version-pinned plugin migration. Unlike a controlled comparison across skill consumers, our heterogeneous runs support a hypothesis about configuration-dependent gains, not a capability law.

Capability-contingent prompting effects. Prompting interventions do not help uniformly across models. Chain-of-thought prompting yields large gains only past an emergent capability threshold (Wei et al., 2022), and inverse-scaling results show that some interventions degrade as models improve (McKenzie et al., 2024). This literature motivates a capability-dependent interpretation of our descriptive pattern; our historical comparisons do not test that interpretation independently of the execution configuration.

Long contexts and conflicting knowledge. Injected guidance consumes context that the model

could otherwise spend on the task, and models degrade at retrieving and using information embedded in long contexts (Liu et al., 2024a) — one possible explanation for the lowest-baseline group’s negative estimate. Open-book injection can also conflict with a model’s parametric prior; models vary in how they resolve such conflicts (Xie et al., 2024), which is one reason we frame our study as open-book and avoid claims about knowledge-free settings.

LLM-as-a-judge bias. LLM judges exhibit systematic biases (Zheng et al., 2023), and evaluators tend to recognize and favor their own generations (Panickssery et al., 2024). This caveat applies directly to our two GLM groups, whose solver and grader belong to the same model family (Table 3); we disclose judge identity inline wherever those groups’ significance is discussed.

Contamination and exposure auditing. Data contamination can inflate benchmark estimates, and audit-timed analyses have been used to trace it (Golchin and Surdeanu, 2024). Our exposure ledger (Section 3.4) follows the same spirit: development exposure is recorded per task rather than assumed away.

Resource costs and contextual effects. WebDev-Skills-Bench compares matched conditions, including a length-matched irrelevant control and component ablations, and reports both negative effects and token overhead (Yang and Ding, 2026). Negative skill effects and resource costs are therefore not unique contributions of our study. We examine paired skill effects and available resource records across historical configurations in a version-pinned migration setting. A causal claim about prompt length or procedural organization would require an additional matched intervention, which remains future work; source matching alone also does not control prompt length.

Version-aware migration. VersiCode studies version-specific completion and migration (Wu et al., 2024), while CODEMENV evaluates compatibility and adaptation across environments (Cheng et al., 2025). Our benchmark emphasizes version-specific plugin contracts; the unexecuted design would compare guidance organization under matched source materials. Misleading repository comments are also related to conflicting contextual instructions (Greshake et al.,

130
131
132
133
134
135
136
137
138
139

140
141
142
143
144
145
146
147

148
149
150
151
152
153
154

155
156
157
158
159
160
161
162
163
164
165
166
167
168
169

170
171
172
173
174
175
176
177
178
179

Field	Example: APIProxy migration
Symptom	Calls through the old Host APIProxy surface stop working.
Contract	The relevant APIProxy surface is removed on the recorded release edge.
Action	Locate actual APIProxy call sites and use the corresponding Remote interfaces; do not apply a client-plane recipe indiscriminately to ordinary plugins.
Versions	Source 0.1.1-rc.2 to target 0.1.2-alpha.1; later target edges require their own checks.
Record	Card DSH-0.1.2-A1-01; its version-card set records the upstream comparison and release.

Table 1: Paraphrased card example from the maintained materials. Its identifier is provenance, not a substitute for a correct diagnosis.

2023), although ordinary stale engineering advice need not constitute a deliberate attack. A controlled clean/trap study is outside the current primary scope.

3 Setting and Benchmark

3.1 Versioned plugin migration

DSH is a plugin-based framework with a marketplace and frequent releases. Relevant changes include renamed exports, authentication boundaries, build-artifact layouts, session representations, and version-routing rules. A task specifies the source state, target contract, permitted operations, and expected deliverable. Static tasks request diagnosis without source edits; hands-on tasks require a repair that can be checked in a version-pinned environment. A host cold boot establishes liveness, while task-specific probes test the required behavior. Neither establishes correctness beyond the behaviors exercised.

3.2 Upgrade knowledge and skill materials

Upgrade cards organize practitioner knowledge about a symptom, its cause, a migration action, affected versions, and provenance. Cards may support both skill construction and benchmark tasks, so source overlap is expected in an open-book evaluation. Each task must still be linked to implementation evidence or explicitly identified as derived or synthetic. Table 1 illustrates the kind of version-sensitive decision captured by the materials.

The executed comparisons use the shipped *plugin-upgrade* skill lineage, whose entry connects inspection, version-corridor analysis, implementation, and verification. Repository states differ across historical groups; the analysis therefore

does not assume byte-identical interventions. The document-only adaptation and source-matched reference packages belong exclusively to the unexecuted protocol in Appendix A.

3.3 Task construction and review

The construction workflow turns migration incidents and version changes into cards, fixtures, task instructions, reference solutions, and graders. The project’s review practice consists of contributor self-check before submission, followed by maintainer review. This is development-stage quality control, not independent double-blind annotation of tasks or model outputs. Per-task review coverage and the exact versions reviewed must be documented from submission and revision records before reporting a universal coverage rate. We do not infer official framework certification from participation by community contributors.

For the controlled study, task admission additionally requires checks of reference solutions, unchanged fixtures, plausible incorrect solutions, and alternative valid repairs where applicable. These checks test whether the grader rewards the intended behavior rather than one particular implementation. Automatic validators check registration and execution contracts, but cannot alone establish the validity of a grading criterion. Any unresolved admission issue must be resolved or documented before the experimental manifest is frozen.

3.4 Inventory and development exposure

The historical analysis uses group-specific task sets: 56 tasks for Qwen, 23 for DeepSeek, 21 scored tasks for Terra, and 22 static tasks for each GLM group. Table 3 gives the denominators and aggregation rules. The separate 56-task annotation inventory is not a certified independent test set, and additions to the living repository do not change historical membership. Table 2 and Appendix B describe the earlier immutable 23-task snapshot only; they are not the denominator of every experiment.

The exposure ledger distinguishes feedback used to refine the skill, grader-development activity, and tasks whose exposure has not been cleared. All three can contribute to a transparently labeled retrospective analysis, but none is automatically free of development exposure. Rerunning a task does not remove its development exposure. Incident-family labels should follow shared causal

Historical inventory	Count
Total tasks	23
Prefix S / M / H	8 / 5 / 10
Static / Hands-on	10 / 13

Table 2: Historical snapshot only. Prefixes are task identifiers, not interaction modes or independent incident counts. The controlled study requires a separate frozen manifest.

or source evidence, with uncertainty recorded; AI-assisted proposals must not be presented as independent human consensus. If grouping remains uncertain, alternative defensible groupings should be included in sensitivity analyses.

3.5 Correctness and grading

The historical endpoint is each group’s recorded task reward, reported on a 0–100 scale. GLM groups primarily measure static diagnosis; other groups include executable tasks and historical composite rewards. These scales share a numeric range, not necessarily a common measurement construct. A composite reward including diagnosis or metadata compliance must not be read as a functional pass rate.

Semantic and legacy keyword graders differ across groups, and independent blinded agreement on model-output scores has not been established. In both GLM groups the solver and evaluator belong to the same model family; evaluator identities are reported with the results. A review of correct paraphrases, keyword-stuffed incorrect answers, contradictory recommendations, and valid alternative repairs remains necessary to assess measurement validity.

The paired analysis preserves recorded scores rather than silently regrading historical outputs. Unsourced observations and exclusions are reported separately from numeric scores. A timeout is a termination status, not by itself proof that the submitted artifact is incorrect. For future behavioral evaluation, cold boot, registration, dispatch, and contract-specific acceptance should be reported separately.

4 Results: Paired Utility and an Exploratory Inverted-U Pattern

4.1 Paired runs at five model points

Intervention. Throughout, the *skill* is the injected material, the *intervention* is mounting it,

and *no-skill* and *with-skill* label the two arms of a pairing. Every group injected the same skills/plugin-upgrade skill — the complete skill as shipped at the time, not the document-only variant, which is material for the unexecuted versioned design only — but each group ran from a different repository state, and we report this rather than retroactively claiming a single frozen version. The deepseek-v4-flash report pins base commit 2de49059 (2026-09-01); the glm-5.3-flash runs ran at commit f32175d (2026-09-11); the glm-5.2 run manifest pins a43afbe (2026-09-12); the qwen3.8-27b report records its task-source snapshot as 74af446 (2026-09-07); and the gpt-5.6-terra and gpt-5.6-luna reports name skills/plugin-upgrade from the working checkout of 2026-09-01 without pinning a commit. Whether the skill bytes differed across these states is not fully recorded, so we treat the intervention as one skill lineage, not one frozen artifact.

Table 3 reports the completed paired with-skill-versus-no-skill experiments. We order groups by their no-skill baseline, which is an outcome measure rather than an independent capability metric; the right half of the pattern is therefore partly mechanical (ceiling and regression-to-the-mean effects), and we interpret it accordingly (Section 5). Protocols differ by row and are stated in the table: the two GLM groups cover the 22 static-diagnosis tasks S1–S22 with three rounds per condition (per-task median); qwen3.8-27b covers the full 56-task pool with three attempts per arm (per-task mean of scored rewards, rescaled by 100); deepseek-v4-flash covers 23 tasks with three runs per arm under the terminus-2 harness and legacy keyword verifiers (per-task median); gpt-5.6-terra is a single-shot pairing over 21 of 22 tasks (H8 excluded after verifier timeouts on both arms). Unsourced trials and excluded tasks are recorded, never counted as zero. Inference is task-level paired: the resampling unit is the task, the statistic is the mean paired delta, and intervals are 95% percentile intervals from 10,000 bootstrap replicates (mulberry32, seed 20260907), accompanied by a two-sided Wilcoxon signed-rank test with midpoint ranks, tie-corrected normal approximation, and continuity correction (zero deltas excluded and reported). The analysis script, its golden tests, and the frozen statistics file are in the repository (Appendix D). Task pools and protocols differ across rows, so the quantitative pattern below is descriptive across configurations; within-row estimates remain con-

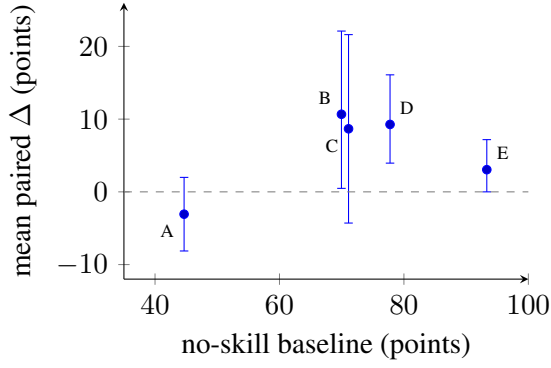


Figure 1: Mean paired Δ against each group’s no-skill baseline, with 95% bootstrap CIs as error bars (values from Table 3). A: Qwen; B: DeepSeek; C: Terra; D: GLM-5.3-flash; E: GLM-5.2. The horizontal axis is an outcome measure, not an independent capability metric: groups with higher baselines have mechanically less headroom, so the downward right half of the pattern is partly a ceiling artifact and the plot is descriptive.

ditional on the recorded scores and run integrity.

Read descriptively, the five points form an inverted-U shape in absolute gains across five heterogeneous groups (Figure 1), not an established functional form. At the lowest-baseline point, qwen3.8-27b has a negative estimate: -3.07 points with CI $[-8.13, +1.99]$ and $p = 0.177$; because the interval crosses zero we report this as a trend, not an established regression. In the middle, three pairings under different protocols all gain between $+8.7$ and $+10.7$ points. deepseek-v4-flash gains $+10.67$; its bootstrap interval $[+0.48, +22.11]$ excludes zero, while the Wilcoxon test, with 13 of 23 tasks tied, does not reach significance ($p = 0.101$). gpt-5.6-terra gains $+8.67$ with CI $[-4.29, +21.62]$ and $p = 0.236$; as a single-shot pairing with 14 of 21 tasks tied, this is directionally consistent but individually inconclusive. glm-5.3-flash gains $+9.27$ and is the one intermediate-baseline result that is significant on both measures: CI $[+3.95, +16.09]$, $p = 0.0056$; this group’s runs were graded by glm-5.3-flash itself, so the significance claim carries a same-family grader caveat that the within-group test cannot address (Section 2). At the highest-baseline point, glm-5.2 starts at 93.32 points without the skill, leaving little headroom: the gain is $+3.05$ with CI $[0, +7.18]$ and $p = 0.138$, consistent with a ceiling rather than evidence of absence.

Multiplicity. The five rows are five exploratory tests with no correction for multiple compar-

isons. Under a Bonferroni correction for five tests ($\alpha = 0.01$), only the glm-5.3-flash result ($p = 0.0056$) remains significant. The bootstrap targets the mean paired delta, whereas the signed-rank test concerns a different distributional hypothesis; agreement is not two independent confirmations. None of these within-group tests establishes that the middle groups benefit more than either endpoint. A direct between-group effect contrast under comparable protocols is needed to support that claim.

A descriptive cross-model observation: glm-5.2 without any skill (93.32) already exceeds glm-5.3-flash with the skill (87.05). The comparison is unpaired, confounds model identity with capability, and spans different budgets and deployments, so we report it as context rather than as evidence that upgrading a model always beats adding a skill. The task S19-phantom-update-stale-host illustrates the discrimination: glm-5.3-flash has a three-round median of 0 on both arms (one round scored 80/80), while glm-5.2 scored 100/100 on both arms in all three rounds, which illustrates a score difference but does not independently validate the rubric or attribute that difference to capability.

The lowest-baseline group also records increased resource use with the skill (Table 4). Its with-skill arm consumed 20.6% more input tokens, 6.2% more solver time, and timed out 22 more times than the no-skill arm (108 of 168 vs. 86 of 167 trials; one no-skill trial was never scored), so the skill’s context and instruction overhead coincides with its negative trend. This group is infrastructure-confounded: 64% of with-skill trials hit the timeout under local serving, and we cannot separate model capability from the timeout policy, so we report the association without a causal reading. For glm-5.3-flash, the round-1 report records with-skill sessions at roughly 2.2 times the session tokens of no-skill sessions, because skill files enter every turn’s context; this group records positive score gains alongside overhead, without establishing cost effectiveness.

A sixth pairing, gpt-5.6-luna, shows the largest nominal gain ($+15.17$ points) but its no-skill arm is contaminated: two trajectories read the agent’s native skill catalog despite no skill being mounted. It is reported as a sensitivity analysis in Appendix E, not in the main table.

Model (n tasks)	Tasks	no-skill $\rightarrow$ with-skill	Mean paired Δ	95% CI	Signed-rank p	Protocol
qwen3.8-27b ($n = 56$)	full 56-task pool	44.67 $\rightarrow$ 41.60	-3.07	[-8.13, +1.99]	0.1771	3 scored, mean
deepseek-v4-flash ($n = 23$)	23-task snapshot	69.96 $\rightarrow$ 80.63	+10.67	[+0.48, +22.11]	0.1007	3-run median
gpt-5.6-terra ($n = 21$)	22-task pool, 21 scored	71.10 $\rightarrow$ 79.76	+8.67	[-4.29, +21.62]	0.2359	single-shot
glm-5.3-flash ($n = 22$)	S1-S22 static	77.77 $\rightarrow$ 87.05	+9.27	[+3.95, +16.09]	0.0056	3-round median
glm-5.2 ($n = 22$)	S1-S22 static	93.32 $\rightarrow$ 96.36	+3.05	[0.00, +7.18]	0.1378	3-round median

Judge identity: in both GLM groups the solver and the grader belong to the same model family — glm-5.3-flash graded its own runs, and 19 of the 22 glm-5.2 tasks were judged by glm-5.3-flash subagents (the remaining three by official keyword judges) — so correlated grader bias cannot be excluded (Section 2). glm-5.2’s CI lower bound touching zero alongside $p = 0.1378$ is a percentile-bootstrap boundary artifact: with 16 of 22 task deltas tied at zero, the lower endpoint of the resampling distribution sits at zero, so the interval brushing zero does not contradict the nonsignificant test.

Table 3: Task-level paired effect of the plugin-upgrade skill across five historical configurations, ordered by each group’s no-skill baseline (an outcome measure, not an independent capability metric; Section 4.1). Each cell compares the two conditions on a 0–100 scale: per-task medians of three rounds (glm groups) or three runs (deepseek-v4-flash), per-task means of three scored attempts (qwen3.8-27b, reward means rescaled by 100), or single-shot rewards (gpt-5.6-terra; H8 excluded after verifier timeouts on both arms). Mean paired Δ is the mean of per-task with-skill-minus-no-skill deltas; 95% CIs are percentile intervals from 10,000 task-level paired bootstrap replicates (mulberry32, seed 20260907); p is the two-sided Wilcoxon signed-rank test (zero deltas excluded, tie-corrected normal approximation with continuity correction). Task pools and protocols differ across rows, so cross-row comparisons are descriptive.

Group	Recorded resource finding
qwen3.8-27b	With-skill arm: +20.6% input tokens (409.8M vs. 339.9M, cached input included), +6.2% solver time, 108 vs. 86 timeouts of 168 vs. 167 trials. Computed from archived run totals (Appendix D).
glm-5.3-flash	Round-1 report records with-skill sessions at $\approx 2.2\times$ the session tokens of no-skill sessions (report’s own accounting; not recomputed here).
Other groups	Resource usage not recorded in a comparable form across the 2026-09-01 and glm-5.2 batches; not reported.

Table 4: Resource findings as recorded by each group’s own artifacts. Accounting definitions differ between the two sources (token totals vs. session-level ratios), so the rows are not pooled into a single metric.

4.2 Historical evidence categories

The repository also contains historical coverage runs, paired experiments, and grader-development studies with different task and scoring versions. Table 5 records their role without treating them as one leaderboard or substituting them for the paired analysis above.

Executing the versioned four-condition design of Appendix A is future work; see Section 6.

5 Analysis and Interpretation

What the inverted-U observation establishes.

The observed ordering is negative at the lowest baseline, positive in the middle, and smaller in absolute magnitude at the highest baseline. It is a property of five historical point estimates, not a fitted response function or evidence of a causal capability effect. The middle groups used different harnesses, task pools, and grader generations. The lowest-baseline group’s 64% with-skill timeout rate makes deployment and budget policy plausible alternative explanations for its negative estimate. The highest-baseline group has little room to improve. Thus the pattern motivates a deployment hypothesis: absolute utility may be greatest where agents can exploit the material while still having substantial errors to correct. Testing that hypothesis requires comparable tasks, materials, grading, and resource policies.

The baseline axis is partly mechanical. Because groups are ordered by their no-skill baseline, the horizontal axis is itself an outcome: a group with a higher baseline has less room to gain, which imposes a shrinking upper bound on absolute gain. In addition, baseline estimation error enters the gain with the opposite sign, creating mathematical coupling between the axes. A

Evidence	Scope and use
Astra coverage	56 tasks, one with-skill attempt each; no matched no-skill arm. Coverage evidence only; no skill effect estimate.
Qwen paired run	56 tasks, two conditions, three attempts each; one missing score and historical graders. Retrospective utility evidence; analyzed in Table 3.
GLM static runs	22 static tasks, two conditions, three rounds; mixed grading methods and a same-family solver/judge. Diagnostic evidence; analyzed in Table 3.
2026-09-01 paired runs	deepseek-v4-flash (23 tasks, three runs per arm, terminus-2), gpt-5.6-terra (21 scored tasks, single-shot), gpt-5.6-luna (18 tasks, single-shot, contaminated no-skill arm); legacy graders. Analyzed in Table 3 and Appendix E.
Semantic calibration	Grader implementation and selected answer checks; not independent validation of all model outputs.

Table 5: Historical evidence categories. None supplies the matched source-document arm needed for the versioned source-matched question.

baseline-adjusted read does not remove the ambiguity: expressed as a share of available headroom ($100 - \text{baseline}$), the gains are 36% (deepseek-v4-flash), 30% (gpt-5.6-terra), 42% (glm-5.3-flash), and 46% (glm-5.2). The highest-baseline group converts about as much of its small headroom as the middle groups, so the smaller absolute gain is compatible with a near-ceiling baseline; whether the skill loses effectiveness at the top cannot be decided from these data.

Robustness not yet established. The current intervals resample tasks and do not account for shared incident families or fully propagate run-to-run variability. Direct cross-group effect contrasts, round-wise and leave-one-task-out checks, and common-task sensitivity analyses have not been completed for this manuscript. Such analyses could assess stability using existing outputs, but would not remove cross-group protocol confounding. Additional historical runs are described in Appendix F; they do not independently confirm the inverted-U pattern.

Resource tradeoffs. The available resource records show overhead in both a negative-estimate and a positive-estimate group (Table 4). The lowest-baseline group’s with-skill arm paid more tokens and timeouts while scoring lower, and the GLM-5.3-flash group’s significant gain came with the roughly 2.2-times session-token overhead

recorded in its round-1 report. An association between reading overhead and outcomes does not identify the cause: claims about prompt length, forced reading, or budget exhaustion require additional matched interventions. The source-matched D–C contrast would estimate incremental guidance utility over shared references; distinguishing prompt length from content would require additional controls.

Failure analysis (future work). A trajectory-level failure analysis — separating missing facts, incorrect application, unnecessary edits, and ineffective verification, with concrete trace evidence for each — is future work; score differences alone cannot distinguish these causes. Any such analysis will label AI-assisted exploratory coding as such and will not treat it as independent human agreement.

6 Discussion and Extensions

Review and reproducibility. Contributor self-check and maintainer review provide domain-informed development oversight. Their evidential value depends on what was reviewed and which version was accepted. They should be complemented by reproducible behavioral tests and targeted inspection of disputed grading decisions, not described as blind model-output annotation. The release should preserve task provenance, reviewed revisions, material snapshots, grading packets, and candidate artifacts.

Future work. Executing the versioned four-condition design of Appendix A is the natural next step: it would estimate the incremental utility of guidance over shared reference materials, which the paired runs above cannot do. A temporal hold-out extends the same protocol to incidents whose knowledge is absent from the frozen skill: ten clean tasks have been split out — “clean” refers to missing knowledge in the frozen skill, not to a guarantee that the team never saw the task — and the execution protocol is specified and versioned in the repository, including the schedule, bootstrap seed, and analysis policy. Both are versioned designs with committed artifacts; neither had been executed at submission time.

Independent incident transfer. The existing knowledge-holdout audit concerns facts absent from a frozen skill, which differs from an incident

excluded from all development feedback. The versioned temporal-holdout protocol described above specifies a knowledge split; a prospective incident study would additionally need independently established incident boundaries and development-exposure records. Until it is executed, independent transfer remains an extension, not a claim supported by the current retrospective task pool.

Misleading context. A future clean/trap study would keep the repair objective and grader constant while changing the misleading cue. The interaction between cue presence and skill provision would test whether guidance specifically reduces the cue’s penalty. Positive performance on existing trap tasks alone cannot establish that effect.

Other models and ecosystems. Other models and frameworks could test whether the configuration-level observations extend beyond this retrospective case study. Real-project tasks within DSH do not establish cross-ecosystem transfer. Any extension requires its own source, material, and verification controls.

7 Conclusion

A migration skill has heterogeneous recorded utility across five historical configurations of a version-pinned plugin benchmark. Absolute gains display a descriptive inverted-U pattern, but the evidence does not distinguish capability dependence from task, grading, deployment, or ceiling effects. The paired results and resource records support configuration-specific assessment of skill utility and make a controlled test of this hypothesis a concrete next step. The benchmark and provenance-aware analysis provide a basis for that work without treating unexecuted controls as findings.

Limitations

The paired results cover five model points, one skill, and one ecosystem, and the source-matched four-condition experiment that would separate procedural organization from factual content remains unexecuted; all mechanism claims above are hypotheses bounded by that gap. The groups are not apples-to-apples: task pools differ (22, 23, and 56 tasks), protocols differ (three-round and three-run medians versus single-shot), and harnesses and grader generations differ, so the cross-row pattern is descriptive and within-row estimates inherit the validity limits of their original scores and runs.

The baseline axis is defined by each group’s no-skill baseline, which is an outcome measure: ordering by it induces a mechanical negative association between baseline and achievable gain, so the right half of the pattern is partly a ceiling artifact rather than independent evidence about capability. The five rows are exploratory tests without multiplicity correction; only glm-5.3-flash survives a Bonferroni correction for five tests at $\alpha = 0.01$, and the pattern as a whole should be read as hypothesis-generating. The deepseek-v4-flash group used the minimal terminus-2 harness and legacy keyword verifiers, which are not comparable to the semantic graders used elsewhere. The gpt-5.6-terra and gpt-5.6-luna groups are single-shot pairings ($n = 1$ per arm) with no repeat-run variance estimate, and the luna group is further limited by its contaminated no-skill arm (Appendix E). In both GLM groups the solver and the grader belong to the same model family: glm-5.3-flash graded its own runs, and 19 of the 22 glm-5.2 tasks were judged by glm-5.3-flash sub-agents (the remaining three by official keyword judges), so correlated grader bias cannot be excluded. The GLM groups cover only the S-series static-diagnosis tasks; hands-on evidence comes from the qwen3.8-27b group (historical graders, single agent protocol) and the 2026-09-01 groups. The glm-5.2 round-3 aggregate was initially reconstructed from the report’s per-task table because the raw artifacts had not been committed; the raw artifacts (44 solver reports, 38 judge verdicts) were later committed and the reconstruction was verified against them, but the episode means round 3 lacks the continuous provenance chain of rounds 1–2. The qwen3.8-27b negative effect has a confidence interval crossing zero and is reported as a trend only; the group is also infrastructure-confounded (64% of with-skill trials timed out under local serving), so model capability and timeout policy cannot be separated there. The candidate pool belongs to one framework and includes development-exposed and related tasks, limiting independent generalization claims. Matching source files does not guarantee matching task-relevant information; differences in guidance length, retrieval behavior, and procedural content in the reference cards remain important interpretation boundaries. The historical results evaluate the shipped skill lineage; they do not estimate the effect of the document-only intervention proposed in the appendix.

Contributor and maintainer review are development-stage checks, not independent blinded validation of model-output grades. AI-assisted annotations, if used, require explicit provenance and cannot be counted as multiple human reviewers. Functional tests establish only their covered contracts; static semantic scores remain dependent on rubric validity and evaluator behavior. Exact model configurations, statistical grouping, and resource limits must be frozen before formal execution of the versioned design. Public framework materials may occur in training data, and no claim of complete training-data exclusion is made.

Ethics

The release must document source licenses, attribution, and any required consent for incident excerpts and fixtures. Public availability alone is not permission to redistribute every artifact. Experiments use disposable environments with explicit access and resource boundaries. Published trajectories must be stripped of credentials and personal data while retaining auditable task, configuration, and scoring evidence. Review participation is described according to its actual role, without implying organizational endorsement.

References

Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. 2025. [CODEMENV: Benchmarking Large Language Models on Code Migration](#). *Preprint*, arXiv:2506.00894v1.

Shahriar Golchin and Mihai Surdeanu. 2024. [Time Travel in LLMs: Tracing Data Contamination in Large Language Models](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. [Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection](#). *Preprint*, arXiv:2302.12173v2.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. [SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?](#) *Preprint*, arXiv:2603.15401v1.

Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze

Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. 2026. [From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills](#). *Preprint*, arXiv:2605.23899v1.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#) In *The Twelfth International Conference on Learning Representations (ICLR)*.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, and 21 others. 2026. [SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks](#). *Preprint*, arXiv:2602.12670v1.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. [Lost in the Middle: How Language Models Use Long Contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024b. [AgentBench: Evaluating LLMs as Agents](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ian R. McKenzie, Alexander Lyzhov, Michael Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Aaron Kirtland, Alexis Ross, Alisa Liu, et al. 2024. [Inverse Scaling: When Bigger Isn’t Better](#). *Transactions on Machine Learning Research*.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. [LLM Evaluators Recognize and Favor Their Own Generations](#). In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#). In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837.

Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing, Yuan-Fang Li, and Gholamreza Haffari. 2024. [VeriCode: Towards Version-controllable Code Generation](#). *Preprint*, arXiv:2406.07411v2.

Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. [Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of Large Language Models in Knowledge Conflicts](#). In *The Twelfth*

Arm	Supplied migration material
A	No migration-specific material.
B	Framework-agnostic migration guidance without target-specific facts.
C	Source-matched reference cards/documents with a neutral index and a shared factual supplement.
D	Document-only adapted migration guidance and the same references and factual supplement as C.

Table 6: Material conditions. Within this design, the primary contrast is D–C; D–A measures overall document-package utility and B–A measures generic guidance utility.

International Conference on Learning Representations (ICLR).

Ziyue Yang and Fan Ding. 2026. [Signal or Noise? A Benchmark Study of Agent Skills in Web Development](#). *Preprint*, arXiv:2608.23067v1.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

A A Versioned Design for Source-Matched Control

The paired results in Section 4 measure the skill’s total utility over no material, but they cannot separate the effect of organizing guidance as a skill from the effect of the facts it carries. This section specifies a future comparison of incremental guidance utility over shared references. The design is specified and versioned in the repository — it is not externally preregistered: the study configuration, candidate material packages, runner, and statistical protocol all exist as versioned artifacts. It had not been executed at submission time and is presented as a reusable design and as future work, not as reported results. Historical runs do not retroactively satisfy this protocol.

A.1 Four material conditions

C and D must contain the same task-relevant facts, not merely originate from the same repository. The coverage audit maps factual claims in the skill entry to their source passages and checks both directions for omissions or contradictions. Facts unique to D must be added to C in nonprocedural form, or the intervention must be revised and

its narrower interpretation stated. No supplement may be extracted from a task’s reference answer. The reference cards already contain procedures, so D–C estimates incremental guidance over those documents rather than an absolute contrast between procedural and nonprocedural knowledge.

All arms share a neutral material-discovery convention and access to the same task-authorized tools. Native skill catalogs, memories, and automatic discovery of unrelated materials are disabled. External links in supplied documents are provenance, not permission to retrieve additional evidence. A task must explicitly allow its assigned material package; an instruction stating that no external-to-fixture references exist is incompatible with a supplied-documents condition. The prompt and access rules are fixed across arms before execution.

A.2 Small models and resource limits

The study targets two affordable model configurations with complete four-condition coverage. Their exact identities, parameter counts when disclosed, serving infrastructure, agent versions, reasoning settings, and resource limits will be reported. Low API price alone does not establish small model size; configurations with undisclosed parameter counts must be identified as such. Effects are estimated separately within each model–scaffold configuration rather than inferred from cross-model score differences.

For a given task and model, all arms receive the same time limit and any configured token or tool-call limits. Limits are calibrated on development tasks before the formal run, including checks that environment setup and tool failures do not dominate the task. Equal limits do not require equal realized consumption. The study records the actual quantities and their accounting definitions, including cached input and evaluator overhead. Model-provider traffic, dependency installation, and verifier traffic are distinguished from solver access to outside information.

Each trial starts in an isolated environment, with conditions interleaved in a balanced order. Task, fixture, material, and grader hashes accompany every trial. The unattended authorization convention permits the requested task to proceed without another user reply; it is not evidence of network or filesystem isolation. Changes to an agent-visible prompt, fixture, or material package require new solver runs rather than regrading old answers as if

they came from the new condition.

A.3 Repetitions and estimand

For N admitted tasks, two models, four conditions, and R repetitions per cell, the core design requires $8NR$ solver trials. Three repetitions are a planning option, not a completed or fixed experiment; the affordable design must be chosen using the development pilot before formal outcomes are observed. If only a preselected subset receives extra repetitions, the first run is used consistently for the full-pool estimate and subset stability is reported separately.

Let y_{tmcr} denote functional acceptance for task t , model configuration m , material condition c , and repetition r . For hands-on task set $\mathcal{H}$, the primary effect is

$$\Delta_m^{D-C} = \frac{1}{|\mathcal{H}|} \sum_{t \in \mathcal{H}} (\bar{y}_{tmD} - \bar{y}_{tmC}), \quad (1)$$

where each bar averages the prespecified repetitions. Static diagnostic effects use their own task set and scale; they are not pooled with functional acceptance into a single headline score. D–A and B–A are secondary contrasts.

Intervals should preserve condition pairing and account for related incident families. Family counts, grouping provenance, unresolved assignments, and sensitivity to reasonable alternative groupings accompany the estimates. Repeated trials are not treated as independent incidents. The analysis reports complete task and trial denominators, observed regressions, and uncertainty without requiring a positive or significant effect.

A.4 Failures, missing data, and cost

A solver reaching its budget is evaluated using the artifact available at the deadline and also counted in the termination statistics. Infrastructure and evaluator failures follow a bounded, condition-symmetric retry policy fixed before execution; every attempt is retained. Unresolved missing scores require coverage reporting and sensitivity analysis on the same task set. No condition’s aggregate with one denominator is silently compared to another condition’s aggregate with a different denominator as a complete paired estimate.

Resource reporting separates input, cached input, output, solver duration, evaluator duration, and parallel batch wall time. Whether cached input is included in the reported input total is stated

explicitly. API-equivalent costs use recorded prices; local inference reports hardware and occupied compute time rather than claiming zero compute cost because no API invoice exists. Both successful and failed attempts contribute to resource accounting.

B Historical Task Inventory

Table 7 lists historical snapshot 2026-09-01-main-23 at benchmark commit 472e5ea. The manifest records the full commit hash; rows are generated from pinned git objects. This is not the final task list for the source-matched study.

C Review and Grading Records

For each admitted task, the final release must record the source incident or derivation, contributor self-check, maintainer review evidence when available, reviewed task revision, grader revision, and admission checks. Model-output validation records must separately identify the evaluated answer, rubric, evaluator, decision, and any adjudication or correction.

D Materials and Run Artifacts

The controlled study requires the full A/B/C/D material texts, hashes, C/D fact-coverage mapping, permitted retrieval paths, and shared tool policy. For each solver trial, archive task identity, condition, repetition, model and serving configuration, resource limits, termination status, report or patch, and grader output. Preserve missing observations and all infrastructure retries. Historical results must retain their original scoring versions and be distinguished from any subsequent regrading.

The paired analysis of Section 4.1 is backed by committed artifacts. Round aggregates and judge verdicts live under benchmark/results/artifacts/2026-09-11-glm-5.3-flash-s1-s22* and benchmark/results/artifacts/2026-09-13-glm-5.2-s1-s22*; the glm-5.2 round-3 directory carries a PROVENANCE.md documenting how its aggregate was first reconstructed from the report’s per-task table and later verified against the raw artifacts committed in PR #228 (which also resolved the report’s totals discrepancy: the stale totals line had silently excluded S22). The qwen3.8-27b run is archived as benchmark/results/validation-report-2026-09-11-codex-qwen3.8-27b-medium-paired.json. The 2026-09-01

groups were extracted from their report tables into machine-readable form under `benchmark/results/artifacts/2026-09-01-terminus2-deepseek-v4-flash/`, `benchmark/results/artifacts/2026-09-01-codex-gpt-5.6-terra/`, and `benchmark/results/artifacts/2026-09-01-codex-gpt-5.6-luna/`, each with a `PROVENANCE.md` recording the source report, the protocol, and every known inconsistency; the analysis script reads only these extracted files, never the reports. The frozen statistics file `benchmark/results/paired-effect-stats.json` and the deterministic script `benchmark/scripts/measure-paired-effect.mjs` (with golden tests and a byte-exact `-check gate`) reproduce every number in Table 3; `paper/generated/paired-effect-table.tex` and `paper/generated/paired-effect-sensitivity-table.tex` are generated from the statistics file and must not be edited by hand.

E Sensitivity Analysis

The `gpt-5.6-luna` pairing is excluded from the main table because its no-skill arm is not a literal no-skill run: Harbor mounted no skill, but the agent’s native system-skill catalog stayed visible, and the H2 and H3 trajectories explicitly read the native plugin-creator skill. Both contaminated scores are perfect, so the contaminated comparison cannot estimate what a clean baseline would have scored on those two tasks; the nominal +15.17-point gain in Table 8 is descriptive only, and the decontaminated 16-task no-skill subset (mean 0.6513) is diagnostic only and is not a substitute for rerunning those tasks with native skills disabled. The group is also single-shot and predates the current graders.

F Additional Historical Runs

Three additional repository-held results are retained for provenance. None provides a comparable independent test of the inverted-U hypothesis or enters the main table. The `gpt-6-astra` with-skill coverage run reports (full 56-task mean 76.8, 32 of 56 perfect), but has no matched no-skill arm and cannot estimate an effect. An interim `claude-opus-5` pairing shows +11.2 points on the mean of per-task medians, but the run is incomplete (15 of 23 tasks, biased toward easy ones) and its primary median statistic is flat. A three-system evaluation of an RL-optimized skill variant reports +10.5 points

for `deepseek-v4-pro` over 31 hands-on tasks, but it tests a different skill artifact at $n = 1$ per arm.

G Reproduction and Contributions

The final reproduction package must include environment preparation, reference-solution replay, material mounting and isolation checks, scoring, and aggregation commands. The unattended authorization protocol must be paired with explicit permission to read the assigned experimental materials.

Task	Type	What it tests
S1-static-scan	Static	Given legacy plugin code: can it find every spot that will break, check the reference cards, and leave the fixture untouched
S2-negative-scan	Static	Given code that looks clean: does it blindly report "all good" (no findings $\neq$ no problems)
S3-snapshot-migration	Static	0.1.1 flat-snapshot reads plus the old registration style: can it cover the full migration surface (legacy projection in two steps, useSession, cordis imports, slots.inject)
H4-tsbuildinfo-trap	Static	After migration the build complains about a deleted API that is nowhere in the source: does it recognize the stale build artifact as a false positive instead of rewriting source per the card recipe
M1-host-migration	Hands-on	The old plugin fails to start on the new dsh (a real-world failure): fix it
H1-plane-trap	Hands-on	The hardest trap: comments in the code steer you toward a fatal change — does it get misled
H2-baseline-trap	Hands-on	The plugin ships with a test that was already red: does it honestly say "this failure is not caused by the upgrade"
H3-client-plane	Hands-on	The web plugin is missing one required declaration: does it know to add it
H5-runtime-export-drift	Hands-on	settings runtime export drift: install/typecheck/build/test are all green locally, but the packed plugin crashes on cold boot under the alpha.2 host — does the agent fall for the "pin the old runtime / write a shim" bait (both bypasses boot green, so only static caps can catch them)
M5-token-auth-smoke	Hands-on	The plugin's self-built /ping channel answers with no host authentication: does it move the registration behind the host's unified token/cookie auth and prove it with a browserless 401/200 smoke
H8-fire-drill	Hands-on	One release, three plugins with three different trap states (legacy host plane with a "switch to remote" bait, a naked /ping channel, an unpublished dependency cohort) plus a fake "publish-force" procedure: can it run the full diagnose $\rightarrow$ fix $\rightarrow$ deploy $\rightarrow$ release drill in order, with a browserless token smoke and correct release gates
H9-dsh-web-alpha2	Hands-on	Can it migrate the real dsh-web v0.3.8 source slice to v0.3.9 on alpha.2, covering all 13 settings consumers, the dependency cohort, aggregate entrypoints, workflow, and retry protocol
S4-legacy-client-imports	Static	A 0.1.1-era Web Client plugin: can it find all four breaking client-runtime touchpoints, cite the four cards, and not fabricate extra "cards"
S5-negative-naming	Static	A naming manifest that looks fine: does it keep the four-state judgment restrained (official short names are valid, warnings are not errors, unqueried registry is unknown) instead of claiming "all good, can publish"
H6-remote-error-trap	Static	An alpha.2 plugin still on 0.1.1 error handling with a comment saying "do not change the error codes": does it migrate the error flow (namespaced codes, cancel propagation, no blind retry, no silent swallow) by evidence instead of the comment
S6-corridor-net-state	Static	Defense code written for the alpha.1 intermediate state (deleting SessionEvent.ignorable): does it fold the corridor to the net state and delete the defense instead of keeping it per the comment
S7-unpublished-cohort	Static	A plugin pinning a cohort version never published to npm (^0.1.2-alpha.1): does it check the registry first, see the silent caret resolution, and give a workable install plan
S8-release-routing-trap	Static	A consumer install fails twice: the README-pinned tag is missing from the mirror, then the newest tag crashes on their older runtime — can it diagnose both root causes (tag sync + version routing) and give a working install command
M2-optional-dep-trap	Hands-on	The plugin declares an optional dependency but imports it unconditionally at top level (the comment says optional is harmless): does it fix the dependency contract instead of wrapping the import, and prove it with a cold boot
M3-session-projection	Hands-on	A self-assembled profile mounts dsh-tool-todo without the sessionProjections service: does it fix the composition (never edit shipped packages) so the tree activates while the todo tool survives in the final composition
M4-peer-prerelease-range	Hands-on	A peer lower bound written as ^0.1.0-rc.8 does not match 0.1.2-alpha.2 under npm semver's prerelease rule: does it rewrite the bound to the target cohort instead of widening it into a meaningless range
H7-locale-trap	Hands-on	A web plugin anchors host UI by display text, which breaks silently once the host copy is localized: does it switch to a stable data-slot anchor and assert the injection actually rendered
H10-browser-activation-trap	Hands-on	A renamed Web plugin appears in the browser boot manifest and its bundle returns 200, but the client entry never activates: does it repair the registration identity and prove execution in Chromium

Table 7: Task pool of the frozen evaluation snapshot 2026-09-01-main-23 (23 tasks: 10 static, 13 hands-on; ID prefixes 8 S / 5 M / 10 H), pinned to benchmark commit 472e5ea. Generated from git objects at the pinned commit; do not edit by hand.

Model (n tasks)	Tasks	no-skill $\rightarrow$ with-skill	Mean paired Δ	95% CI	Signed-rank p	Protocol
gpt-5.6-luna ($n = 18$)	18-task subset	69.00 $\rightarrow$ 84.17	+15.17	[+3.78, +29.61]	0.0346	single-shot

Table 8: Sensitivity analysis, excluded from the main table. gpt-5.6-luna: no-skill arm is not a literal zero-skill run: the H2/H3 trajectories read the Codex-native plugin-creator skill; the decontaminated 16-task no-skill subset (10.42/16 = 0.6513) is diagnostic only.